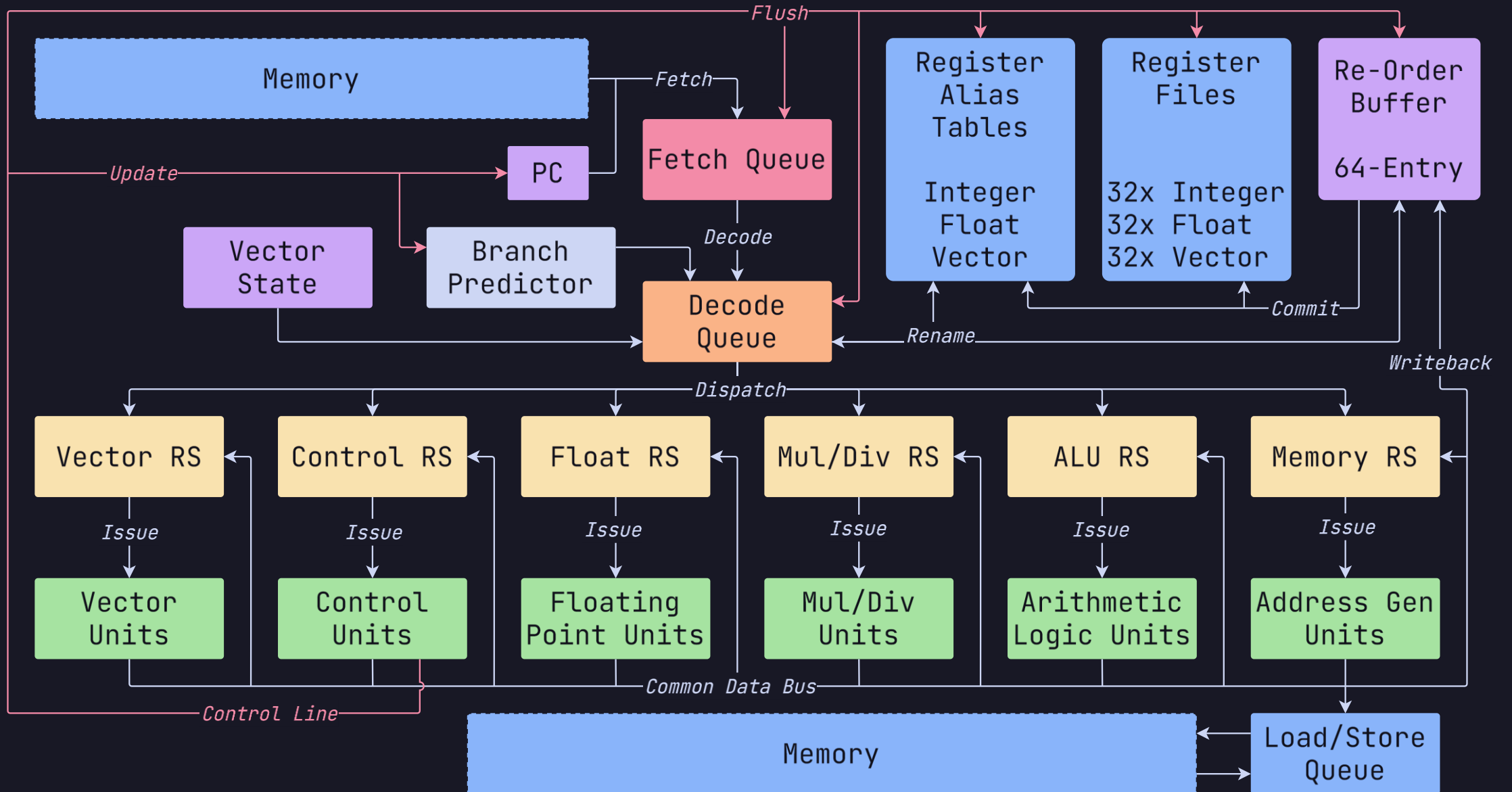


RISC-V Super-Scalar Out-of-Order Simulator

Finn Cooper – sd22404



Features

Instruction Set

RV32I base ISA - missing FENCE, EBREAK, ECALL
RV32M mul/div extension - full implementation
RV32F float extension - missing frm, fcsrs, fflags
RV32V vector extension - LMUL=m1, SEW<=32, no tail/mask

Features

Seven-stage pipeline:
fetch, decode, dispatch, issue, execute, writeback, commit

Modelled multi-cycle instruction latency
Out-of-order execution via reservation stations
In-order commit via re-order buffer
Speculative execution via branch prediction
CDB-like broadcast to reservation stations
Load-store queue with relaxed memory consistency

Branch Prediction

Static taken / not taken
Backwards taken, forwards not taken
Dynamic two-bit saturating counter

Memory

Von Neumann Memory Architecture

Non-overlapping loads bypass pending older stores
Overlapping loads forward bytes from older stores

Configuration

Configurable fetch/issue/commit width
Configurable number of each execution unit type
Configurable reservation station capacity
Configurable load-store queue capacity
Configurable vector length (128, 256, 512)
Configurable branch prediction policy

Terminal UI

Scrollable disassembled instruction view
Scrollable register content view
Scrollable stack content view
Step/run/help keybinds
Optional headless mode

Benchmarks

Dot

Performs the dot product of two 1024-length 32-bit integer vectors. Stresses ALU/MUL throughput, simple loop branch prediction, sequential loads.

Vector variant tests repeated vector-int multiply-accumulate and vector-int reduction.

Matmul

Matrix-multiplies two 16x16 32-bit integer matrices. Stresses multiply throughput and loop overhead.

Vector variant tests repeated vector-int multiply-accumulate and vector loads/stores.

SAXPY

Computes $[y = ax + y]$ on two 1024-length 32-bit floating-point vectors. Stresses repeated fused multiply-add, load-store bandwidth and predictable loops.

Vector variant tests repeated vector-float fused multiply-add.

Quicksort

Recursively sorts a 128-length 32-bit integer array. Stresses data-dependent branching, function calls via recursion, and irregular memory access.

Gaussian

Performs 3x3 convolution over an 18x18 32-bit floating-point matrix. Stresses nested loops, floating-point accumulation and structured memory access.

Vector variant tests repeated convolution style vector-float multiply-accumulate operations.

Fibonacci

Recursively calculates the 14th Fibonacci number. Stresses branch prediction, function call overhead, and pipeline stalling.

Experiment One – Pipeline Width

Hypothesis

Increasing pipeline width and number of execution units will lead to increased IPC across all benchmarks.

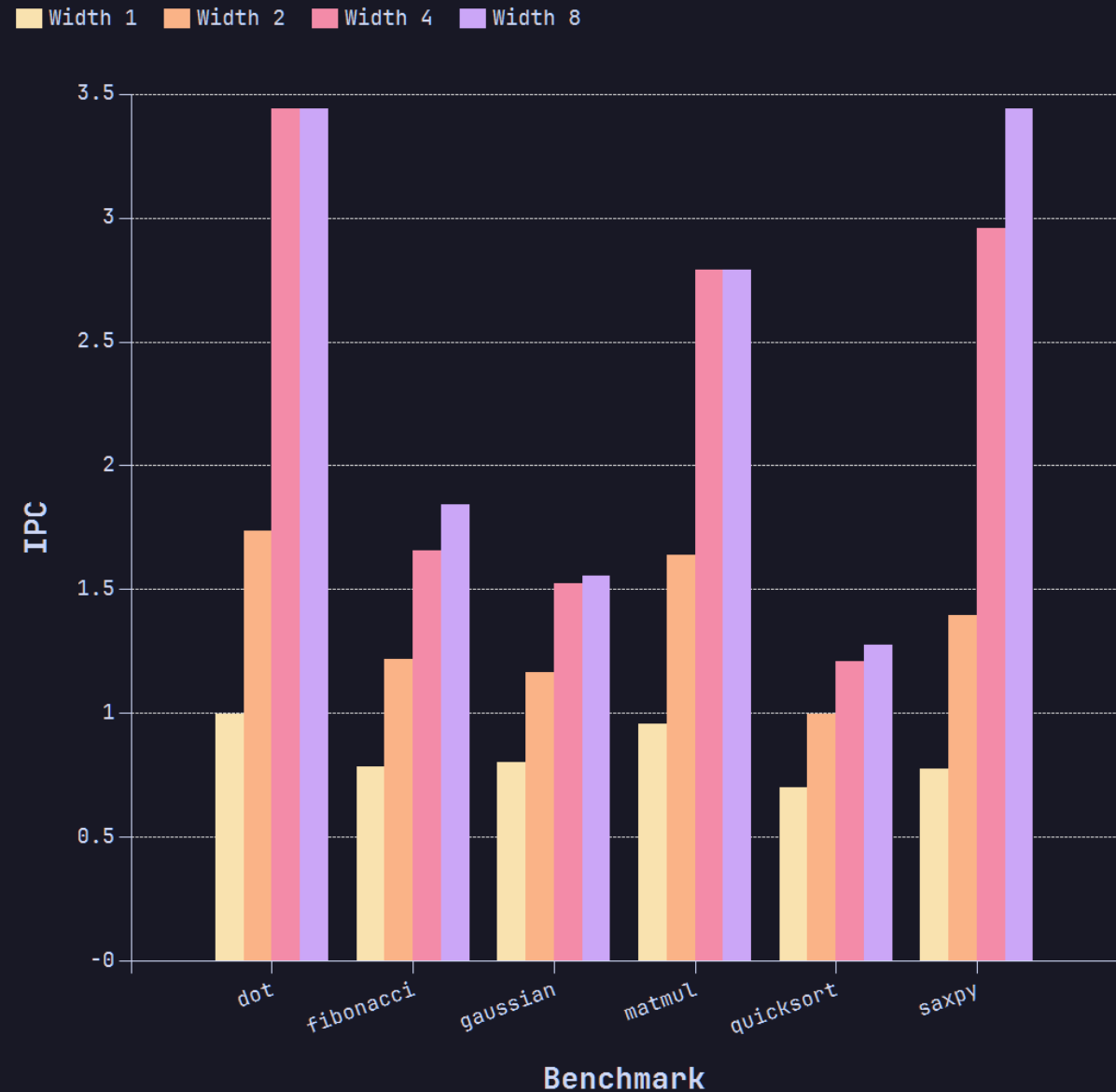
Experiment

All benchmarks were simulated with four different configurations: 1-, 2-, and 4-wide fetch/issue/commit widths. The number of each execution unit type was scaled 1:1 with pipeline width. Reservation station capacities were scaled 4:1 with width. Instructions per cycle was calculated in each case.

Results

Average IPC increases from 0.836 at width-1 to 2.391 at width-8. All benchmarks besides dot product make use of the full 8-width pipeline and show improvement there.

8-width shows a very marginal improvement over 4-width. Suggests that there are other bottlenecks that surface, like branch misses or memory dependencies. Fibonacci and quicksort scale poorly because of their recursive and therefore branch-heavy nature.



Experiment Two – Branch Prediction Policies

Hypothesis

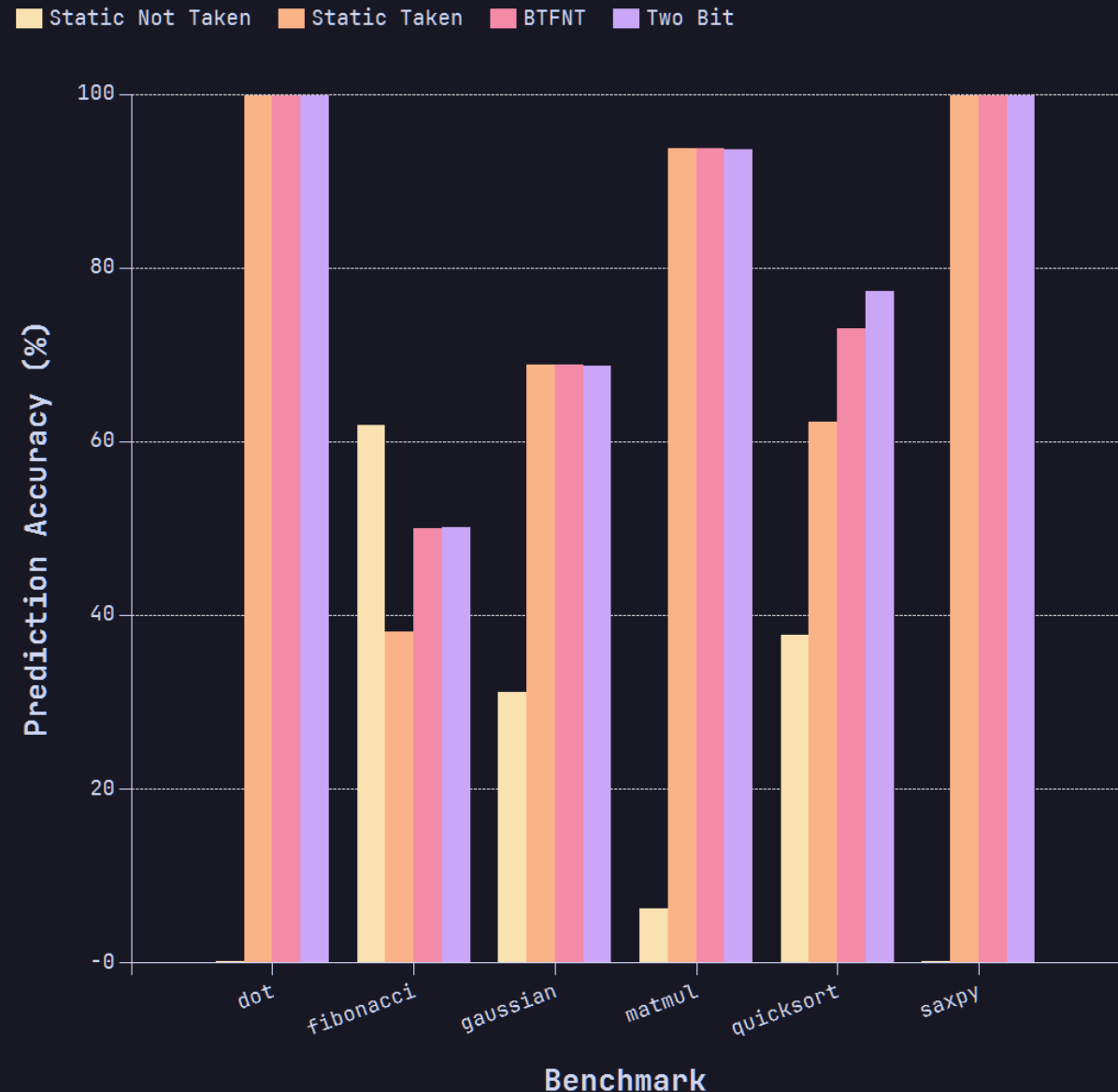
Dynamic branch prediction will show an improvement in prediction accuracy over static policies in most cases.

Experiment

All benchmarks were simulated with four different branch prediction policies: three static (always taken, never taken, backwards taken forwards not taken), and one dynamic (two-bit saturating counter). Prediction accuracy (correctly predicted divided by total branch count) was calculated in each case.

Results

Best policy is two-bit with 81.59% average accuracy, narrowly followed by BTFNT with 80.91%. Benchmarks are loop-heavy so static-taken and BTFNT perform well. Static not taken performs terribly in most cases, especially dot, saxpy, and matmul as loops are almost always taken. It performs well in fibonacci due to its mixed taken / not taken paths. Quicksort benefits the most from dynamic branch prediction as it is much less structurally predictable.



Experiment Three - Vectorisation

Hypothesis

Increasing vector length will improve performance by reducing the number of cycles taken to complete.

Experiment

The four SIMD-friendly benchmarks were executed in four different configurations: a scalar baseline (2-width), and vectorised with VLEN=128, VLEN=256, and VLEN=512. Ratio of number of cycles taken to complete vs the baseline was calculated in each vectorised case.

Results

All benchmarks show the cycle speedup roughly doubling at each vector length step. Average speedup is 3.89x at 128-bit, 7.349x at 256-bit and 14.508x at 512-bit.

Benchmarks reach a peak of almost 16x speedup at VLEN=512, due to processing 16 elements (VLEN / 32-bit) per iteration instead of just 1 in the scalar case.

